

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

Nombre: Kenneth Isaías Yahid López Castillo

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

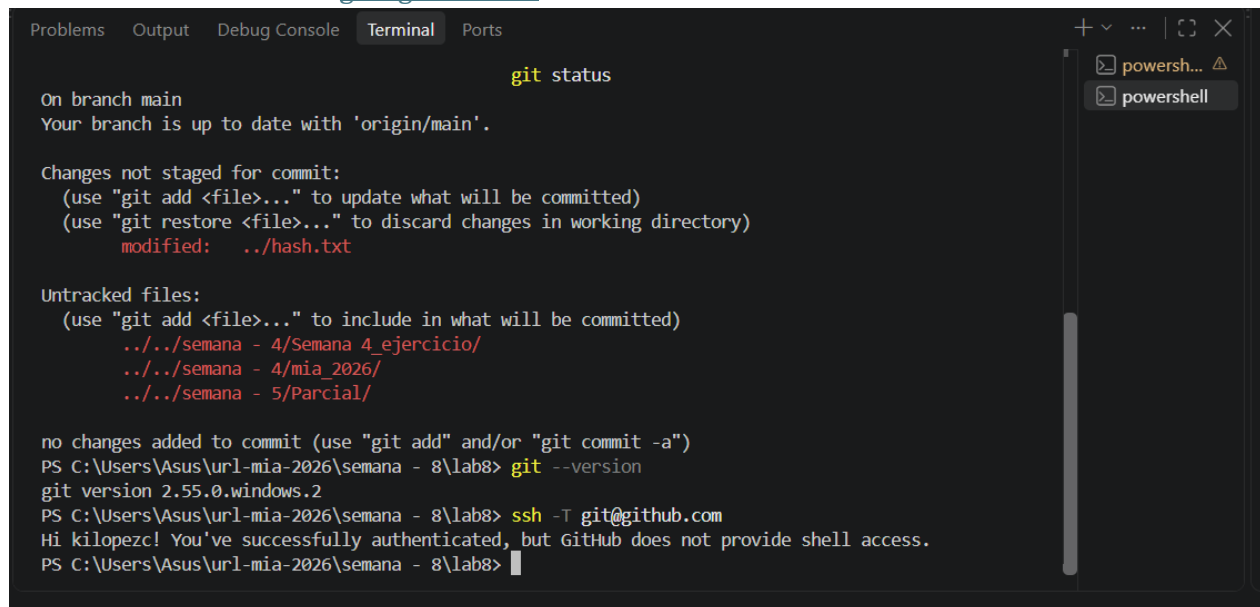
Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH

Verifica Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`



```
git status

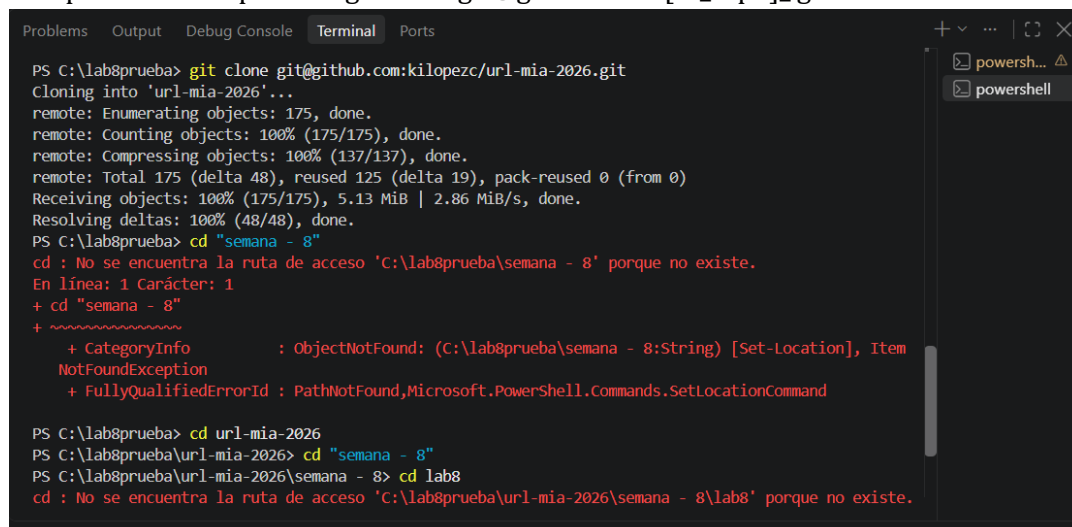
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ../hash.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../../semana - 4/Semana 4 ejercicio/
        ../../semana - 4/mia_2026/
        ../../semana - 5/Parcial/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Asus\url-mia-2026\semana - 8\lab8> git --version
git version 2.55.0.windows.2
PS C:\Users\Asus\url-mia-2026\semana - 8\lab8> ssh -T git@github.com
Hi kilopezc! You've successfully authenticated, but GitHub does not provide shell access.
PS C:\Users\Asus\url-mia-2026\semana - 8\lab8>
```

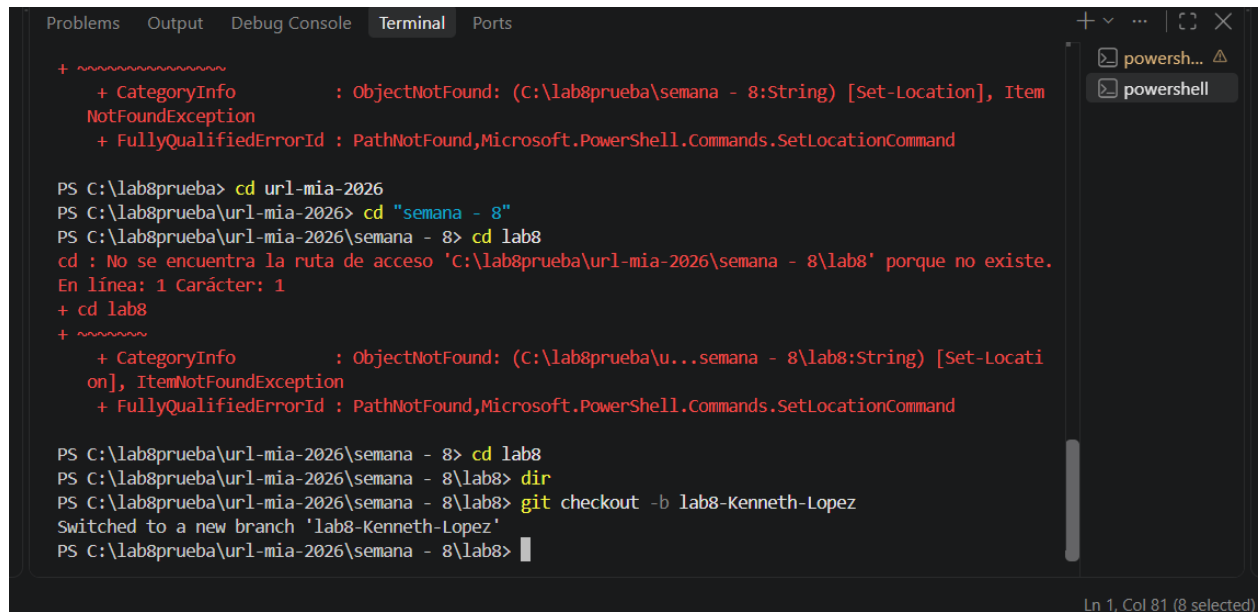
Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo. Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`



```
PS C:\lab8prueba> git clone git@github.com:kilopezc/url-mia-2026.git
Cloning into 'url-mia-2026'...
remote: Enumerating objects: 175, done.
remote: Counting objects: 100% (175/175), done.
remote: Compressing objects: 100% (137/137), done.
remote: Total 175 (delta 48), reused 125 (delta 19), pack-reused 0 (from 0)
Receiving objects: 100% (175/175), 5.13 MiB | 2.86 MiB/s, done.
Resolving deltas: 100% (48/48), done.
PS C:\lab8prueba> cd "semana - 8"
cd : No se encuentra la ruta de acceso 'C:\lab8prueba\semana - 8' porque no existe.
En línea: 1 Carácter: 1
+ cd "semana - 8"
+ ~~~~~
+ CategoryInfo          : objectNotFound: (C:\lab8prueba\semana - 8:String) [Set-Location], Item
  NotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\lab8prueba> cd url-mia-2026
PS C:\lab8prueba\url-mia-2026> cd "semana - 8"
PS C:\lab8prueba\url-mia-2026\semana - 8> cd lab8
cd : No se encuentra la ruta de acceso 'C:\lab8prueba\url-mia-2026\semana - 8\lab8' porque no existe.
```

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`



```
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\lab8prueba\semana - 8:String) [Set-Location], Item
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\lab8prueba> cd url-mia-2026
PS C:\lab8prueba\url-mia-2026> cd "semana - 8"
PS C:\lab8prueba\url-mia-2026\semana - 8> cd lab8
cd : No se encuentra la ruta de acceso 'C:\lab8prueba\url-mia-2026\semana - 8\lab8' porque no existe.
En línea: 1 Carácter: 1
+ cd lab8
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\lab8prueba\url-mia-2026\semana - 8\lab8:String) [Set-Location], Item
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

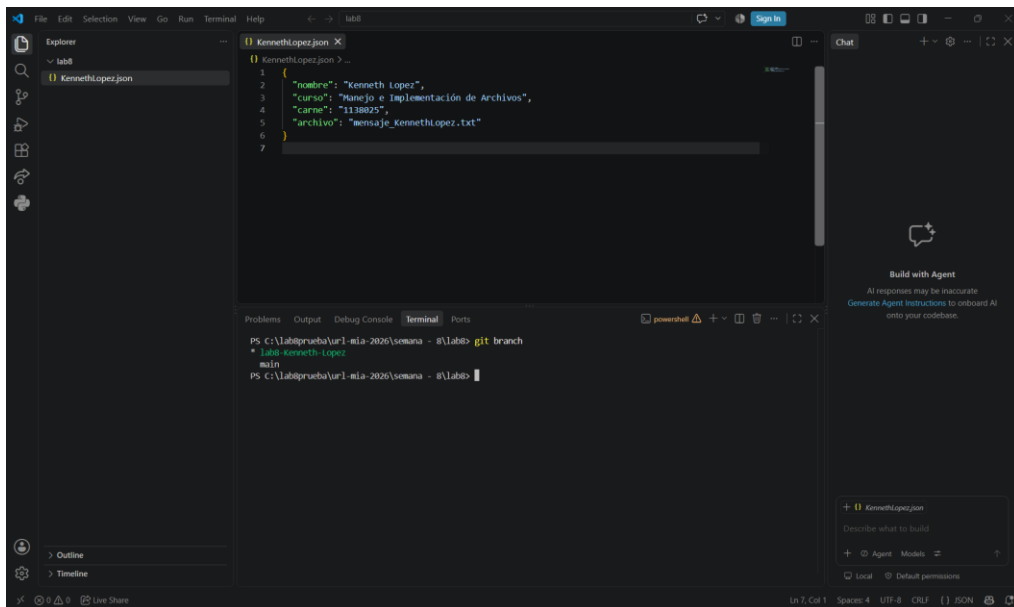
PS C:\lab8prueba\url-mia-2026\semana - 8> cd lab8
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> dir
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> git checkout -b lab8-Kenneth-Lopez
Switched to a new branch 'lab8-Kenneth-Lopez'
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> |
```

2. Crear un archivo personalizado

Crea un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: mensaje_LuisRojas.json Contenido:

```
{
  "nombre": "TU NOMBRE",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "[tu_carne]",
  "archivo": "mensaje_[Nombre_apellido].txt"
}
```



```
1 {
2   "nombre": "Kenneth Lopez",
3   "curso": "Manejo e Implementación de Archivos",
4   "carne": "118025",
5   "archivo": "mensaje_kenneth.lopez.txt"
6 }
7

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> git branch
* lab8-Kenneth-Lopez
main
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> |
```

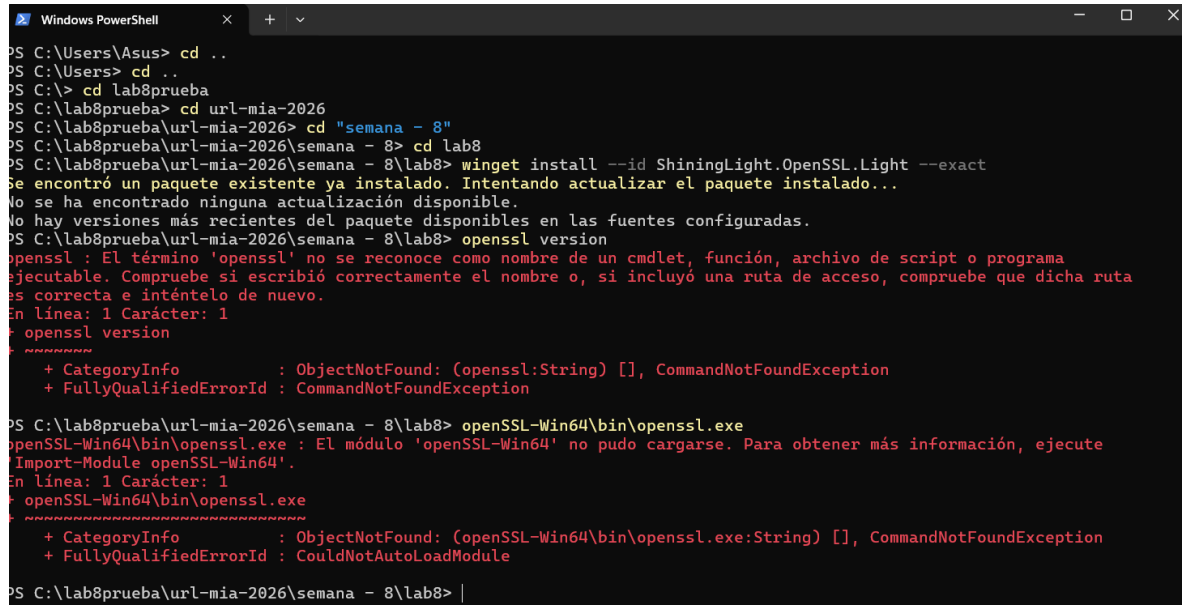
3. Encriptar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión .enc

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

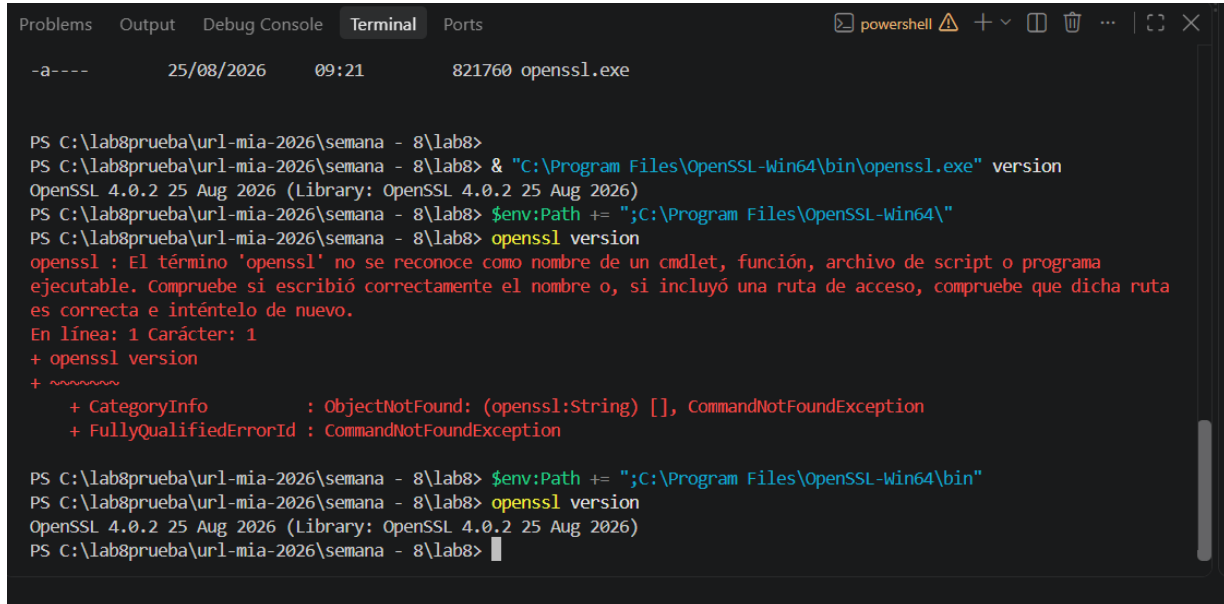
Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact` Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.



```
PS C:\Users\Asus> cd ..
PS C:\Users> cd ..
PS C:\> cd lab8prueba
PS C:\lab8prueba> cd url-mia-2026
PS C:\lab8prueba\url-mia-2026> cd "semana - 8"
PS C:\lab8prueba\url-mia-2026\semana - 8> cd lab8
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> winget install --id ShiningLight.OpenSSL.Light --exact
Se encontró un paquete existente ya instalado. Intentando actualizar el paquete instalado...
No se ha encontrado ninguna actualización disponible.
No hay versiones más recientes del paquete disponibles en las fuentes configuradas.
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl version
openssl : El término 'openssl' no se reconoce como nombre de un cmdlet, función, archivo de script o programa
ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta
es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ openssl version
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (openssl:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl-Win64\bin\openssl.exe
openssl-Win64\bin\openssl.exe : El módulo 'openssl-Win64' no pudo cargarse. Para obtener más información, ejecute
'Import-Module openssl-Win64'.
En línea: 1 Carácter: 1
+ openssl-Win64\bin\openssl.exe
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (openssl-Win64\bin\openssl.exe:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoLoadModule

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> |
```



```
-a--- 25/08/2026 09:21 821760 openssl.exe

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8>
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> & "C:\Program Files\OpenSSL-Win64\bin\openssl.exe" version
OpenSSL 4.0.2 25 Aug 2026 (Library: OpenSSL 4.0.2 25 Aug 2026)
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> $env:Path += ";C:\Program Files\OpenSSL-Win64\"
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl version
openssl : El término 'openssl' no se reconoce como nombre de un cmdlet, función, archivo de script o programa
ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta
es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ openssl version
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (openssl:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> $env:Path += ";C:\Program Files\OpenSSL-Win64\bin"
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl version
OpenSSL 4.0.2 25 Aug 2026 (Library: OpenSSL 4.0.2 25 Aug 2026)
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> |
```

3.2 Comprueba instalación

`Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

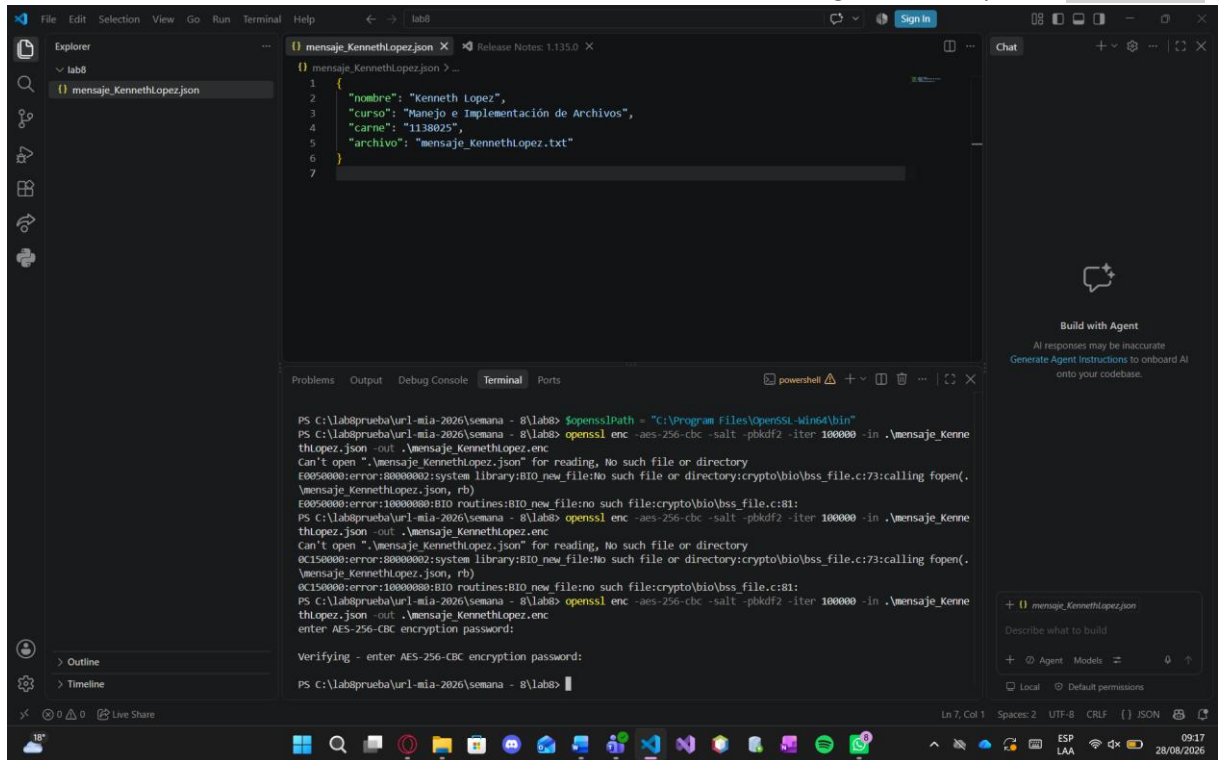
3.3 Encriptar tu archivo

Comprueba dónde quedó instalado: `Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`

[Environment]::SetEnvironmentVariable("Path", \$env:Path + ";" + \$opensslPath, "User")

Actualiza el PATH de esta terminal inmediatamente: \$env:Path += ";C:\Program Files\OpenSSLWin64\bin"



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a file named `mensaje_KennethLopez.json`. The main editor displays the content of this file, which is a JSON object with the following fields:

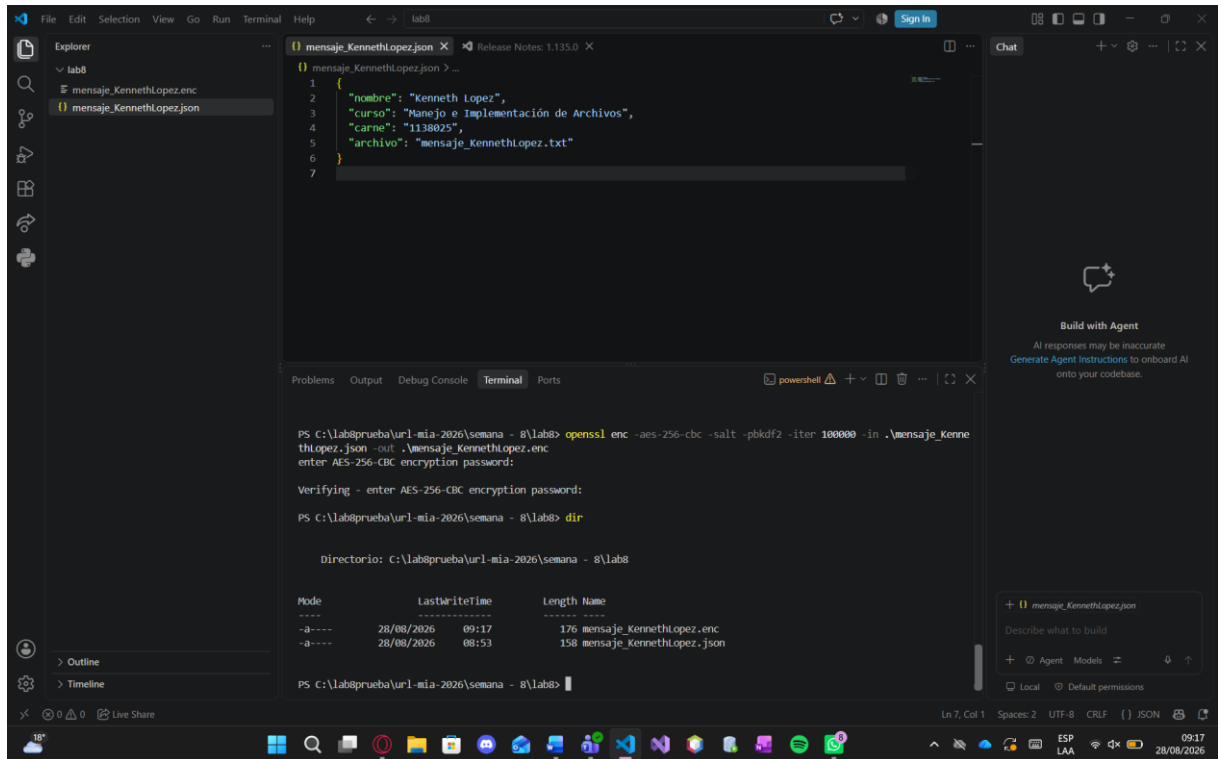
```
1 {
2   "nombre": "Kenneth Lopez",
3   "curso": "Manejo e Implementación de Archivos",
4   "carne": "1138025",
5   "archivo": "mensaje_KennethLopez.txt"
6 }
7
```

The Terminal pane at the bottom shows a PowerShell session. The user has set the `opensslPath` environment variable and attempted to run the following command:

```
PS C:\lab\prueba\url-mia-2026\semana - 8\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_KennethLopez.json -out .\mensaje_KennethLopez.enc
```

The command failed with the following error messages:

```
Can't open ".\mensaje_KennethLopez.json" for reading, No such file or directory
E0050000:error:80000002:system library:BIOS_new_file:No such file or directory:crypto\bio\bss_file.c:73:calling fopen(.
.\mensaje_KennethLopez.json, rb)
E0050000:error:10000000:BIOS routines:BIOS_new_file:No such file:crypto\bio\bss_file.c:81:
PS C:\lab\prueba\url-mia-2026\semana - 8\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_Kenne
thLopez.json -out .\mensaje_KennethLopez.enc
Can't open ".\mensaje_KennethLopez.json" for reading, No such file or directory
0C150000:error:80000002:system library:BIOS_new_file:No such file or directory:crypto\bio\bss_file.c:73:calling fopen(.
.\mensaje_KennethLopez.json, rb)
0C150000:error:10000000:BIOS routines:BIOS_new_file:No such file:crypto\bio\bss_file.c:81:
PS C:\lab\prueba\url-mia-2026\semana - 8\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_Kenne
thLopez.json -out .\mensaje_KennethLopez.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
PS C:\lab\prueba\url-mia-2026\semana - 8\lab8>
```



The screenshot shows the Visual Studio Code interface after the encryption command has been successfully executed. The Explorer pane on the left now shows two files: `mensaje_KennethLopez.enc` and `mensaje_KennethLopez.json`. The main editor displays the content of `mensaje_KennethLopez.json`, which is the same JSON object as in the previous screenshot.

The Terminal pane at the bottom shows the PowerShell session. The user has entered the encryption password and the command has completed successfully. The user then runs the `dir` command to list the contents of the current directory:

```
PS C:\lab\prueba\url-mia-2026\semana - 8\lab8> dir
```

The output of the `dir` command is as follows:

```
Directorio: C:\lab\prueba\url-mia-2026\semana - 8\lab8

Mode                LastWriteTime         Length Name
----                -
-a-----         28/08/2026    09:17           176 mensaje_KennethLopez.enc
-a-----         28/08/2026    08:53           158 mensaje_KennethLopez.json
```

The PowerShell prompt is now at `PS C:\lab\prueba\url-mia-2026\semana - 8\lab8>`.

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

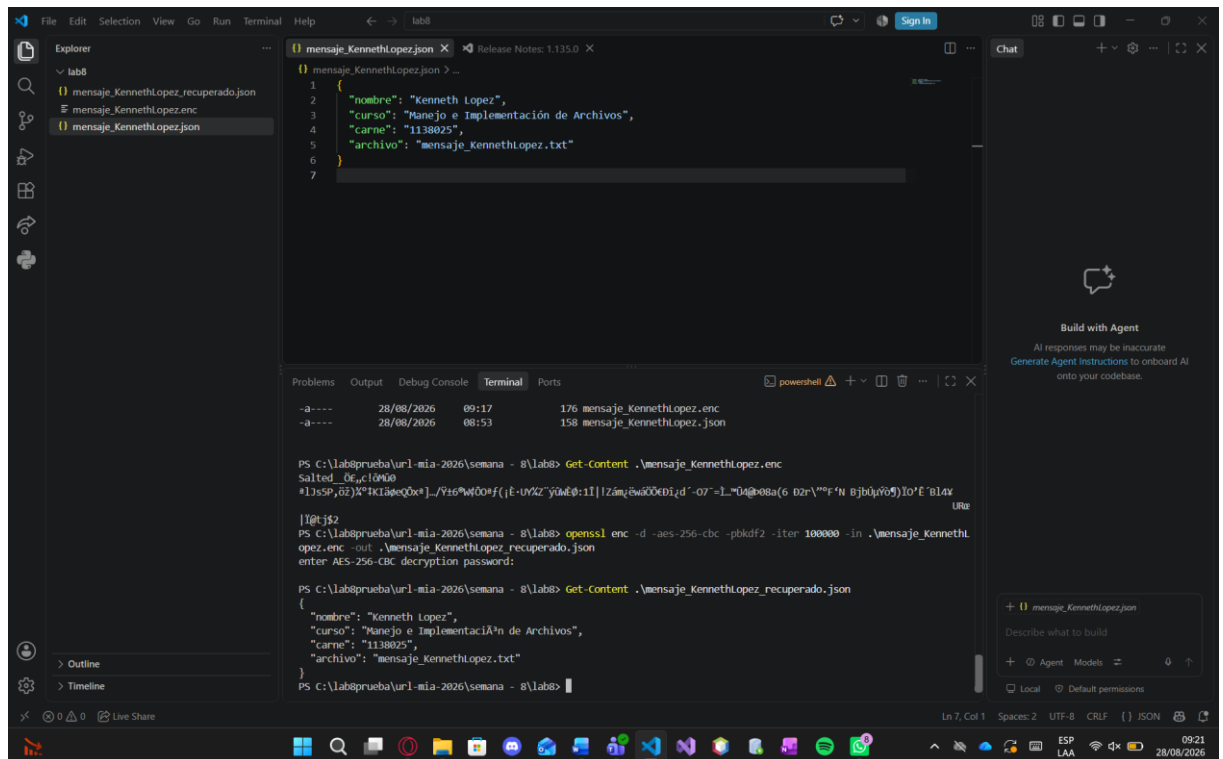
Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO** escribas la contraseña en este laboratorio ni la subas a GitHub.

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc` ¿Qué observaste?



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a file named `mensaje_KennethLopez.json`. The main editor shows the content of this JSON file:

```
1 {
2   "nombre": "Kenneth Lopez",
3   "curso": "Manejo e Implementación de Archivos",
4   "carne": "1138025",
5   "archivo": "mensaje_KennethLopez.txt"
6 }
7
```

The Terminal pane at the bottom shows the following commands and output:

```
PS C:\lab8\prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez.enc
Salted__0E...cl0W00
#1j5SP,0Z)S"K1a8eq0x]_Y16*W00+f(|E-UVZ"Y0d8:1I||Zám9w00Ed;d"-07"-L_0d@0sa(6 02r\""F"N Bjb0y09)I0'E'8L4v
URR

[]@tj52
PS C:\lab8\prueba\url-mia-2026\semana - 8\lab8> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_KennethLopez.enc -out .\mensaje_KennethLopez_recuperado.json
enter AES-256-CBC decryption password:

PS C:\lab8\prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez_recuperado.json
{
  "nombre": "Kenneth Lopez",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "1138025",
  "archivo": "mensaje_KennethLopez.txt"
}
```

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

`openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out .\mensaje_TuNombre_recuperado.json`

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

```
Problems Output Debug Console Terminal Ports
-a----      28/08/2026      09:17      176 mensaje_KennethLopez.enc
-a----      28/08/2026      08:53      158 mensaje_KennethLopez.json

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez.enc
Salted_ÖE,,c!öMü0
#1Js5P,öZ)%*KIäðeQÖxª].../ÿ±6*WfÖ0ªf(¡È·UYZ"ÿüwÈø:1Ï|!Zám;ëwäöÖēDî¿d´-07´=Ï...™Ü4@p08a(6 D2r\”°F‘N BjbüµŸöŹ)Ïo’É`B14¥
URø

|Ï@tj$2
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_KennethLopez.enc -out .\mensaje_KennethLopez_recuperado.json
enter AES-256-CBC decryption password:

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez_recuperado.json
{
  "nombre": "Kenneth Lopez",
  "curso": "Manejo e ImplementaciÃ³n de Archivos",
  "carne": "1138025",
  "archivo": "mensaje_KennethLopez.txt"
}
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> 
```

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json` ¿Cuál es el HMAC-SHA256 obtenido?

```
Problems Output Debug Console Terminal Ports
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez.enc
Salted_ÖE,,c!öMü0
#1Js5P,öZ)%*KIäðeQÖxª].../ÿ±6*WfÖ0ªf(¡È·UYZ"ÿüwÈø:1Ï|!Zám;ëwäöÖēDî¿d´-07´=Ï...™Ü4@p08a(6 D2r\”°F‘N BjbüµŸöŹ)Ïo’É`B14¥
URø

|Ï@tj$2
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_KennethLopez.enc -out .\mensaje_KennethLopez_recuperado.json
enter AES-256-CBC decryption password:

PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> Get-Content .\mensaje_KennethLopez_recuperado.json
{
  "nombre": "Kenneth Lopez",
  "curso": "Manejo e ImplementaciÃ³n de Archivos",
  "carne": "1138025",
  "archivo": "mensaje_KennethLopez.txt"
}
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_KennethLopez.json
HMAC-SHA2-256(.\mensaje_KennethLopez.json)= 4c1f47e6de632377f97af5c88cc2f953ee65be3848a947e9e859e49ba176ae82
PS C:\lab8prueba\url-mia-2026\semana - 8\lab8> 
```

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?
Por confidencialidad. El archivo cifrado (AES-256-CBC) es ilegible sin la clave derivada de la contraseña, así que aunque alguien lo robe, no puede recuperar el contenido original sin la contraseña.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

Porque GitLab es un repositorio compartido, cualquiera con acceso al repositorio podría leerla, y quedaría expuesta permanentemente en el historial de Git. Si la contraseña se filtra, el cifrado del archivo pierde todo su sentido. La contraseña debe manejarse fuera del control de versiones.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

- **SHA-256**=es una función hash pura, toma datos y produce un resumen fijo. Cualquiera puede calcularlo, no requiere clave. Sirve para verificar integridad, pero no autenticidad, alguien podría modificar el archivo y recalcularlo sin que te des cuenta.
- **HMAC-SHA256** combina SHA-256 con una clave secreta. Solo quien conoce la clave puede generar el HMAC correcto. Esto agrega autenticación además de integridad, asegura no solo que el archivo no cambió, sino que fue generado por alguien que tiene la clave compartida.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.